Deep Learning

Flickr Image classification

Project report

Course Instructor: Shemair Williams

Group 1:

Elie MAKOUL
Michael AOUN
Joe TOUMA
Lavanya B VASANT

April 2026

Executive summary

This project focuses on the development and evaluation of deep learning systems for image classification using the Flickr dataset. The dataset contains 7,643 images categorized into five distinct classes: animals, objects, people, scenery, and others. The primary objective was to transition from basic neural architectures to advanced models to identify the most efficient strategy for high-accuracy classification.

Experimental Progression & Methodologies

The research followed a structured pipeline, starting with data preprocessing where images were resized (typically 128x128 pixels) and split into training (6,114 images) and validation (1,529 images) sets.

The models evolved through three main phases:

Phase 1: Multi-Layer Perceptrons (MLP) Two architectures, a shallow network and a deep bottleneck network were tested. While they provided a baseline, their performance was capped at approximately 54% validation accuracy because flattening 2D images into 1D arrays destroys essential spatial hierarchies.

Phase 2: Custom Convolutional Neural Networks (CNN) To capture spatial features, several CNN variations were developed.

- Baseline CNN: Suffered from severe overfitting due to a 125M+ parameter bottleneck caused by the Flatten layer.
- Optimization (GAP2D): Replacing the Flatten layer with GlobalAveragePooling2D reduced the parameter count by 99.9%. This proved to be the most effective custom modification, achieving a 61.00% validation accuracy.

Phase 3: Transfer Learning (MobileNetV2) The final phase utilized a pre-trained MobileNetV2 backbone. By leveraging features already learned from the ImageNet dataset, this model significantly outperformed all custom architectures.

Table of Contents

I. Data Preparation and Preprocessing .. … .. … .. … .. … .. … .. … .. … .. … .. … .. … .. … . 1

II. Multi-Layer Perceptron (MLP) Networks.. … .. … .. … .. … .. … .. … .. … .. … .. … .. … . 1

II.I MLP 1 (Shallow Architecture.. … .. … .. … .. … .. … .. … .. … .. … .. … .. … .. … .. . 1

II.II MLP 2 (Deep Bottleneck Architecture).. … .. … .. … .. … .. … .. … .. … .. … .. . 2

III. Convolutional Neural Networks: Model Optimization and Interpretability.. … .. … .. 3

III.I Baseline CNN Architecture.. … .. … .. … .. … .. … .. … .. … .. … .. … .. … .. … .. . 3

III.II Experiment 1: CNN with Dropout.. … .. … .. … .. … .. … .. … .. … .. … .. … .. … .. . 5

III.III Experiment 2: CNN with Batch Normalization.. … .. … … … ... . … 6

III.IV Experiment 3: CNN with Global Average Pooling 2D.. … .. .. … .. … .. … . 7

IV. Model interpretability technology.. … .. … .. … .. … .. … .. … .. … .. … .. … .. … .. … .. . 9

V. Transfer Learning (MobileNetV2) … .. … .. … .. … .. … .. … .. … .. … .. … .. … .. … .. … 11

VI. Conclusion.. … .. … .. … .. … .. … .. … .. … .. … .. … .. … .. … .. … … … .. … .. … 12

List of Figures

Figure 1: MLP 1 vs. MLP 2 Training Accuracy and Loss … .. … .. … .. … .. … .. … .. … 2

Figure 2: Baseline CNN Overfitting Curves … .. … .. … .. … .. … .. … .. … .. … .. … .. 4

Figure 3: Impact of Dropout on CNN Training… .. … .. … .. … .. … .. … .. … .. … .. … 5

Figure 4: CNN with Batch Normalization Accuracy/Loss … .. … .. … .. … .. … .. … 6

Figure 5: CNN with Global Average Pooling 2D Performance … .. … .. … .. … .. … .. . 7

Figure 6: Effects of Data Augmentation on Training… .. … .. … .. … .. … .. … .. … 8

Figure 7: Grad-CAM Visualizations for Model Focus… .. … .. … .. … .. … .. … .. … 9

Figure 8: Occlusion Sensitivity Heatmaps … .. … .. … .. … .. … .. … .. … … … .. … .. … .. . 10

Figure 9: MobileNetV2 Transfer Learning & Fine-Tuning Results.. … .. … .. … .. … .. …11

List of Tables

Table 1: Performance Comparison of Custom CNN Experiments… .. … … … .. … .. … .. 9

Table 2: Comprehensive Summary of All Evaluated Models… .. … … … .. … .. … .. .. … 12

APPENDIX

Unscaled Performance Graphs… .. … .. … .. … .. … .. … … … .. … .. … .. … .. .. … .. … ii

I. Data Preparation and Preprocessing

Before defining and training the neural networks, a data preparation pipeline was established to ensure reproducibility and optimal model performance:

- **Reproducibility:** Global random seeds were set to guarantee consistent data splitting, weight initialization, and outputs across all models.
- **Data Splitting:** Using CSV metadata, the dataset was partitioned into **6,114 training** and **1,529 validation** images.
- **Standard Processing (MLP & Custom CNN):** Images were resized to **128x128 pixels**, normalized to a [0, 1] range (rescaling factor of **1./255**), and processed using a **batch size of 32**. For MLPs, these 2D images were additionally flattened into 1D arrays.
- **Data Augmentation:** To improve CNN robustness, spatial transformations were applied dynamically during training via preprocessing layers built directly into the model architecture.
- **Transfer Learning Processing:** MobileNetV2 required a distinct pipeline; its images were resized to **160x160 pixels** and normalized using the model's native preprocess input function.

I. Multi-Layer Perceptron (MLP) networks

Following the data preparation and normalization steps, where all images were resized to 128 × 128 × 3, two different MLP architectures were defined and trained to establish a baseline for classification.

II.I MLP 1 (shallow architecture) The first model was designed as a shallow network containing only one wide hidden layer. Because the dense layer is fully connected to the flattened high-dimensional input, this model resulted in a massive parameter count.

architecture pipeline:

- Flatten(input_shape=(128, 128, 3))
- Dense(512, activation='relu')
- Dense(5, activation='softmax')

Trainable parameters: 25,168,901

II.II MLP 2 (deep bottleneck architecture) The second model was designed as a deeper network with three hidden layers. By utilizing narrower layers, the architecture successfully reduced the total number of parameters while adding depth. Additionally, a dropout layer was introduced to provide regularization.

architecture pipeline:

- Flatten(input_shape=(128, 128, 3))
- Dense(256, activation='relu')
- Dropout(0.3)
- Dense(128, activation='relu')
- Dense(64, activation='relu')
- Dense(5, activation='softmax')

Trainable parameters: 12,624,645

training and conclusion

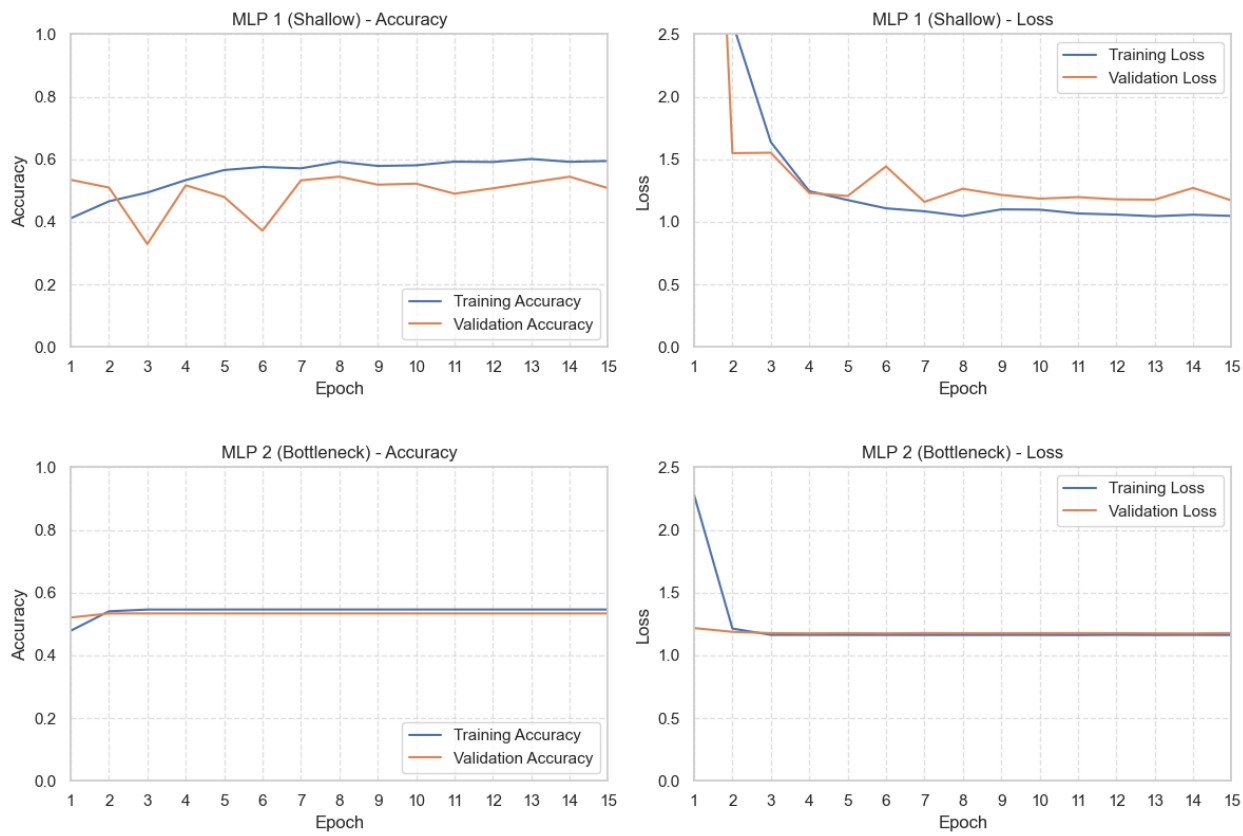


Figure 1:Training and validation accuracy (left) and loss (right) for MLP 1 and MLP 2 over 15 epochs.

Based on the final training and validation metrics shown in the graphs, **MLP 2 was selected as the superior model**.

Although MLP 1 had fewer hidden layers, it possessed nearly double the parameters of MLP 2 due to its wide single hidden layer. Consequently, MLP 1 showed stronger fitting on the training data (achieving a higher training accuracy) but suffered from significant overfitting, as seen by the widening gap between its training and validation curves.

In contrast, MLP 2 generalized much better to unseen data . It achieved a slightly higher final validation accuracy. The narrower layers and the inclusion of dropout effectively regularized the network, proving that validation performance is the critical metric for model selection.

Ultimately, these results highlight a fundamental limitation of MLPs: flattening 2D images destroys spatial hierarchies, which heavily restricts model performance for complex image classification tasks. This limitation directly motivates the transition to Convolutional Neural Networks (CNNs) in the next phase of the project.

II. Convolutional neural networks: model optimization and interpretability

Summary of CNN experiments and hyperparameters

To overcome the spatial limitations of the Multilayer Perceptron (MLP) models, the project transitioned to Convolutional Neural Networks (CNNs). Across the CNN experiments, a consistent set of hyperparameters was established to ensure a fair comparison:

Input Dimensions: Images were maintained at a height and width of 128x128 pixels.

- **Batch Size**: 32
- **Maximum Epochs**: 25
- **Learning Rate**: 0.001
- **Loss Function**: Categorical Cross entropy.

Regularization (Early Stopping): To prevent excessive overfitting and save computational resources, an Early Stopping callback was implemented with a patience of 5 epochs

III.I Baseline CNN architecture

The first CNN developed (Baseline CNN) was designed to extract basic spatial hierarchies before passing the features to dense layers for classification

architecture pipeline:

- Conv2D(32 filters, 3x3 kernel, activation='relu')
- MaxPooling2D(pool_size=2x2, strides=1, padding='same')
- Conv2D(64 filters, 3x3 kernel) followed by a LeakyReLU activation
- MaxPooling2D(pool_size=2x2, strides=1, padding='same')
- Flatten()
- Dense(128 units, activation='relu')
- Dense(5 units, activation='softmax')
- Best epoch 1

Trainable Parameters: 125,980,357

Baseline performance and conclusion

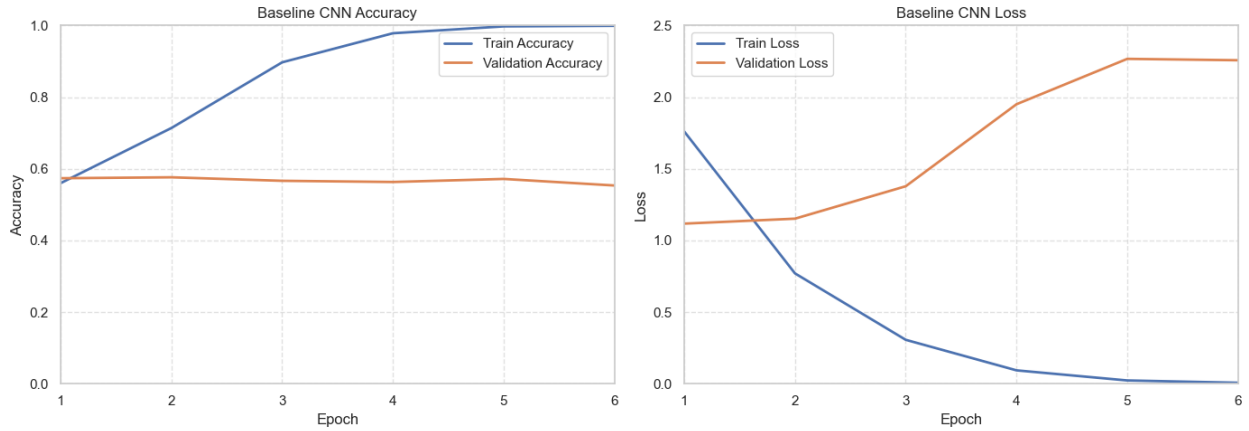


Figure 2: Training and validation accuracy and loss for the Baseline CNN.

The baseline CNN suffered from a massive parameter count (over 125 million), primarily due to the Flatten layer bridging the high-dimensional convolutional feature maps to the dense layers.

As a result, the learning curves exhibited severe signs of overfitting. As the number of epochs increased, the training accuracy climbed to nearly 100%, while the validation accuracy steadily declined from an initial peak at **epoch 1** from roughly 57.36% down to 55.33%. A similar diverging trend was observed in the loss metric, where training loss approached zero while validation loss increased. Early stopping was successfully triggered to halt this degradation.

The final evaluation of this baseline model yielded a training accuracy of 64.00% (loss: 0.87) against a validation accuracy of 57.36% (loss: 1.12). While the validation accuracy showed a slight improvement over the MLP models, the extreme overfitting clearly indicated that the network was memorizing the training data rather than generalizing. This necessitated further model optimization and architectural improvements

III.II Experiment 1: CNN with dropout

To address the severe overfitting observed in the baseline model, the first optimization step involved introducing regularization through a Dropout layer.

Architecture Modifications: A Dropout(0.5) layer was inserted immediately after the first fully connected Dense layer. The rest of the architecture remained identical to the baseline.

- Conv2D(32 filters, 3x3 kernel, activation='relu')
- MaxPooling2D(pool_size=2x2, strides=1, padding='same')
- Conv2D(64 filters, 3x3 kernel) followed by a LeakyReLU
- MaxPooling2D(pool_size=2x2, strides=1, padding='same')
- Flatten ()
- Dense (128 units, activation='relu')
- **Dropout (0.5)** *(New addition)*
- Dense (5 units, activation='softmax')

Trainable Parameters: 125,980,357 (Remains unchanged, as Dropout does not add trainable parameters).

performance and observations

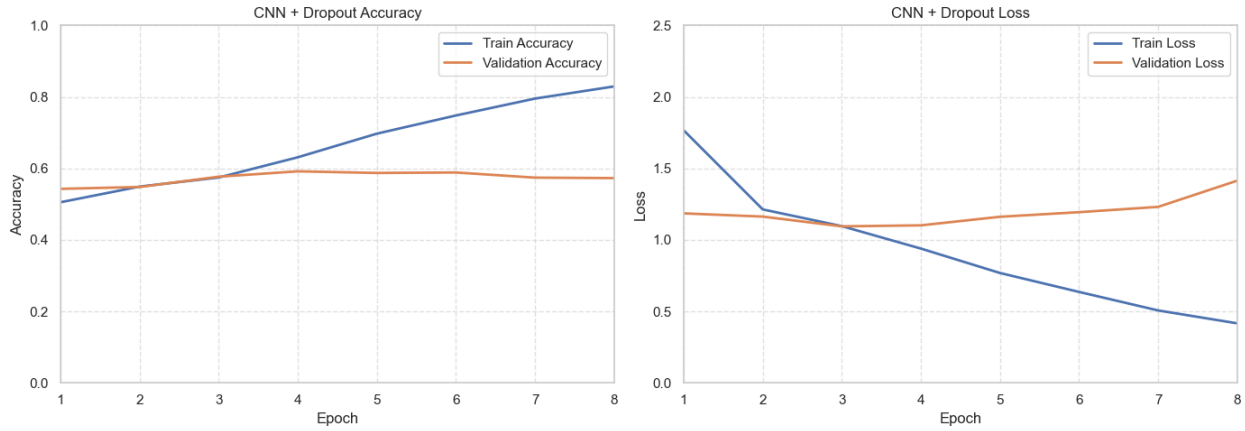


Figure 3: Training and validation accuracy and loss for the CNN with Dropout.

Adding the Dropout layer resulted in a clear improvement in both accuracy and loss compared to the baseline CNN. The model's training was halted by early stopping at epoch 8 with **best epoch =3** .

The final evaluation yielded a training accuracy of 64.00% (loss: 0.85) and a validation accuracy of 57.68% (loss: 1.09). Based on the learning curves, introducing a 50% dropout rate successfully reduced the degree of overfitting seen in the baseline model, as the gap between training and validation metrics narrowed.

However, despite this improvement, clear signs of overfitting were still present as epochs increased. The massive parameter count from the Flatten layer remained a bottleneck, indicating that dropout alone was insufficient to fully regularize the network.

III.III Experiment 2: CNN with batch normalization

For the second optimization experiment, the previously added Dropout layer was removed, and the focus shifted to stabilizing and accelerating the learning process by using Batch Normalization within the feature extraction phase.

- **Architecture Modifications**: Two Batch Normalization layers were introduced directly into the convolutional part of the network. The first was added immediately after the initial convolutional layer (post-ReLU activation), and the second was placed after the second convolutional layer (post-LeakyReLU activation).
    - Conv2D(32 filters, 3x3 kernel, activation='relu')
    - **BatchNormalization()** *(New addition)*
    - MaxPooling2D(pool_size=2x2, strides=1, padding='same')
    - Conv2D(64 filters, 3x3 kernel) followed by a LeakyReLU
    - **BatchNormalization()** *(New addition)*
    - MaxPooling2D(pool_size=2x2, strides=1, padding='same')
    - Flatten()
    - Dense(128 units, activation='relu')
    - Dense(5 units, activation='softmax')
- **trainable parameters**: 125,980,741 (A slight increase from the baseline due to the learnable parameters within the Batch Normalization layers).

performance and observations

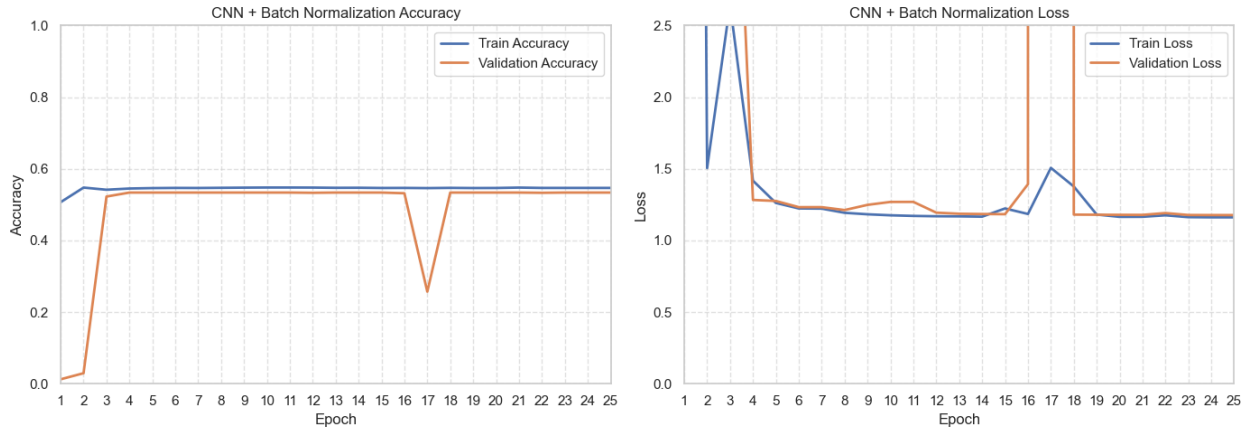


Figure 4: Training and validation accuracy and loss for the CNN with Batch Normalization.

Restoring model weights from the end of the best epoch which is 25., yielding a final training accuracy of 54.66% (loss: 1.16) and a validation accuracy of 53.37% (loss: 1.18).

While Batch Normalization successfully neutralized the extreme overfitting seen in the baseline, keeping training and validation metrics closely aligned, it caused a substantial drop in overall accuracy compared to the Dropout experiment. Ultimately, though the model was stabilized and prevented from memorizing data, it struggled to learn complex patterns due to the critical architectural flaw of flattening over 125 million parameters.

III.IV Experiment 3: CNN with global average pooling 2d

To permanently resolve the massive parameter bottleneck caused by the Flatten layer, the final custom CNN experiment utilized a GlobalAveragePooling2D layer.

Architecture modifications: The Flatten layer was removed entirely and replaced with GlobalAveragePooling2D, which averages each feature map into a single value before passing it to the dense layers.

- Conv2D (32 filters, 3x3 kernel, ReLU)
- MaxPooling2D (2x2, strides=1, padding='same')
- Conv2D (64 filters, 3x3 kernel) followed by LeakyReLU
- MaxPooling2D (2x2, strides=1, padding='same')
- Dense (128 units, ReLU)
- Dense (5 units, softmax)

Trainable parameters: 28,357 (~99.9% reduction from the baseline).

Performance and observations

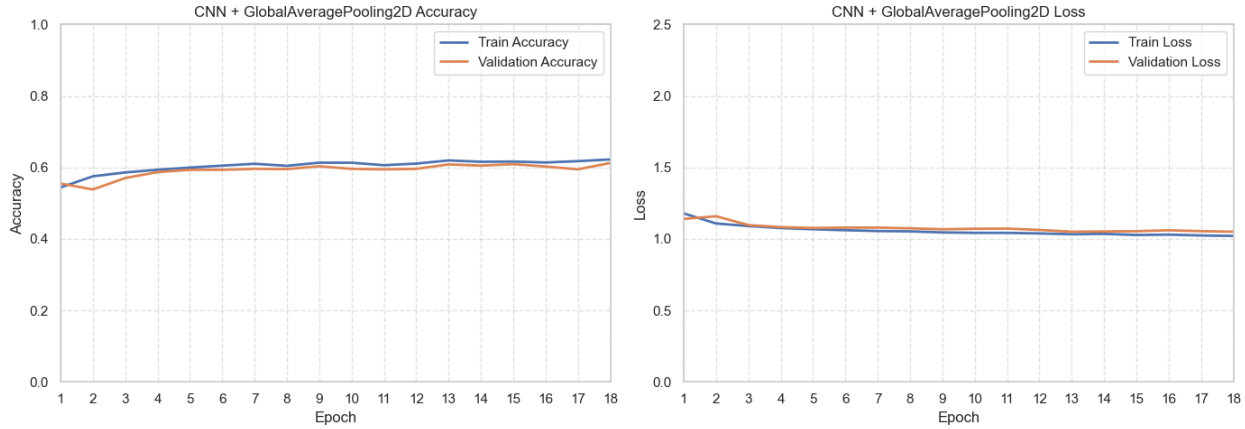


Figure 5:Training and validation accuracy and loss for the CNN with Global Average Pooling.

This architectural change yielded the best custom model performance in the project. Early stoppage at epoch 18 **(peaking at epoch 13).**

The final evaluation resulted in a training accuracy of 61.94% (loss: 1.02) and a validation accuracy of 60.82% (loss: 1.05). Replacing the flatten layer with global average pooling drastically decreased overfitting compared to the baseline and the previous two experiments, forcing the model to learn more generalized features.

Data Augmentation : To further improve the robustness of the best-performing custom architecture (CNN + GlobalAveragePooling2D), data augmentation was introduced.

Procedure: Rather than augmenting images beforehand, three spatial transformation layers were added directly to the beginning of the model architecture to augment data on-the-fly during training:

- RandomFlip("horizontal")
- RandomRotation(0.1)
- RandomZoom(0.1)

architecture pipeline: The augmented model fed directly into the GAP2D architecture established in Experiment 3. **Trainable parameters**: 28,357 (Augmentation layers do not add learnable parameters).

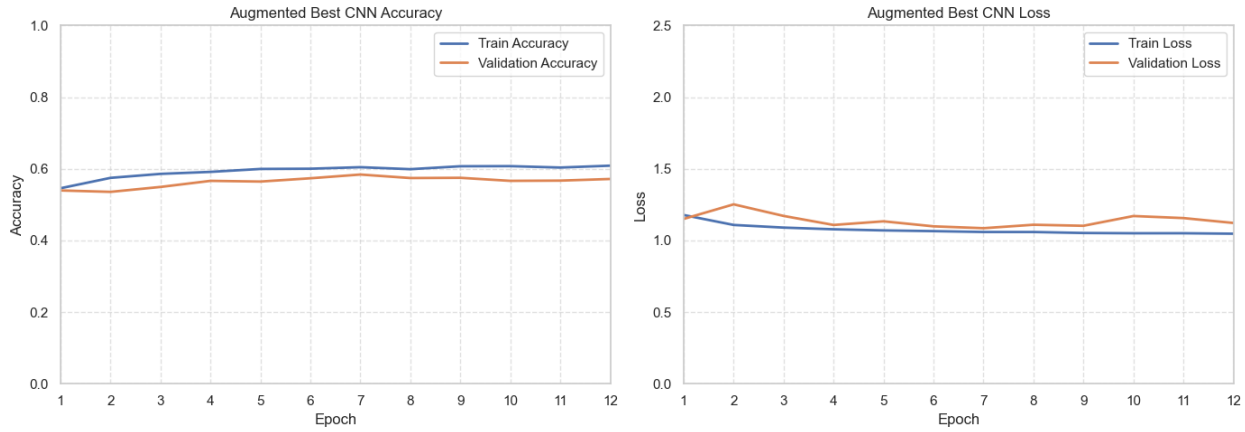


Figure 6: Training and validation accuracy and loss for the augmented CNN.

This architectural change yielded the best custom model performance in the project. The model successfully trained for nearly all 12 epochs **(peaking at epoch 7).**

The final evaluation resulted in a training accuracy of 60.00% (loss: 1.06) and a validation accuracy of 58.40% (loss: 1.09).

	Model	# Final Train Accuracy	# Final Val Accuracy	# Final Train Loss	# Final Val Loss
0	Baseline CNN	0.64	0.57	0.87	1.12
1	CNN + Dropout	0.64	0.58	0.85	1.09
2	CNN + Batch Normalization	0.55	0.53	1.16	1.18
3	CNN + GlobalAveragePooling2D	0.62	0.61	1.02	1.05
4	Augmented Best CNN	0.6	0.58	1.06	1.09

Table 1: Comparison table of different CNN experiments

We can conclude that the best model is CNN with GAP2D. Achieving a final validation accuracy of 61% and final validation loss of 1.05.

III. Model interpretability techniques

To evaluate whether the best custom model (CNN + GlobalAveragePooling2D) focuses on meaningful subjects rather than background noise, visual interpretability techniques were applied.

Grad-CAM

Grad-CAM generates heatmaps highlighting the specific image regions that most strongly influence the model's classification.

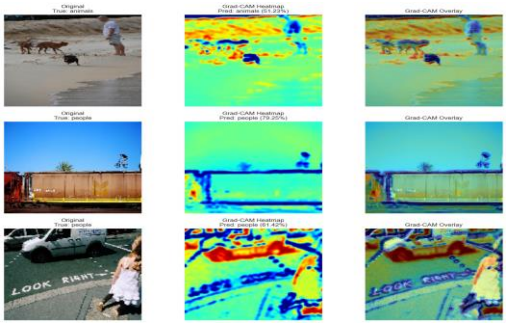


Figure 7: Grad-CAM heatmaps for three sample images showing the model's areas of focus.

Observations & discussion:

Dispersed attention: Across the sample images (predicted "Animals" at 51.23% confidence, and "People" at 79.25% and 61.42% confidence), the network's focus rarely isolates the true target subject. Instead, attention goes into the surroundings.

Shortcut learning: Because the model does not strictly focus on the target objects, it is likely relying on background environmental cues—or "shortcuts"—to infer the class, rather than learning definitive visual traits.

Resolution limitations: Grad-CAM computes from the final convolutional layer, which has a much lower spatial resolution than the original image. Consequently, it only provides a rough area of importance. This inability to yield a pixel-perfect explanation becomes particularly obvious with smaller objects.

Occlusion sensitivity

The second interpretability technique applied was Occlusion Sensitivity. This perturbation-based method systematically masks different areas of an image to measure how the model's prediction confidence changes. In the resulting maps:

- **Hot colors:** Indicate highly important regions. Masking these areas causes a significant drop in prediction confidence.
- **Cool/dark colors:** Indicate less important regions where masking has little to no effect on the prediction.

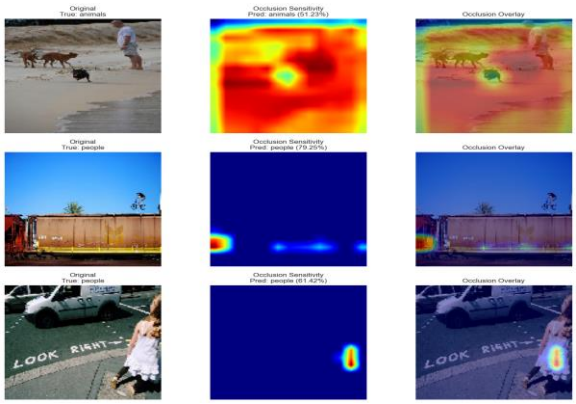


Figure 8: Occlusion Sensitivity maps for three sample images.

Observations & discussion:

Broad and non-specific focus (Image 1): While the model correctly identifies the true target (the animals/dogs), it simultaneously assigns high importance to the person and the surrounding environment, resulting in a very broad and unspecific focus map

Irrelevant highlighting (Images 2 & 3): In the second image, the model places intense importance on a very small, irrelevant area , completely missing the actual subject of the picture. A similar reliance on background features is seen in the third image.

Contextual reliance: Corroborating the Grad-CAM findings, these occlusion maps confirm that the model heavily relies on contextual and background information using environmental shortcuts rather than focusing on defining visual characteristics of the true target objects.

IV. Transfer learning (MobileNetV2)

The final phase of part 1 leveraged Transfer Learning using the MobileNetV2 architecture. By utilizing a model pre-trained on the massive ImageNet dataset, the network had already learned to extract robust visual features such as edges, textures, shapes, and complex object parts ,providing a highly optimized starting point for our specific classes.

Architecture & initial training

A custom classification head was built on top of the pre-trained MobileNetV2 backbone:

- **Input layer:** (160, 160, 3)
- **Feature extractor:** MobileNetV2 Backbone
- **Pooling:** GlobalAveragePooling2D (Converts feature maps into a vector)
- **Regularization:** Dropout(0.2)
- **Output layer:** Dense(5 units, softmax)
- **Parameters:** 2,264,389

During the initial training phase (with the base weights frozen), early stopping triggered at **epoch 6 with best epoch = 3**, achieving a training accuracy of 76.53% (loss: 0.6804) and a validation accuracy of 74.62% (loss: 0.7419).

Fine-tuning & final performance

To maximize performance, the base model was subsequently unfrozen and fine-tuned.

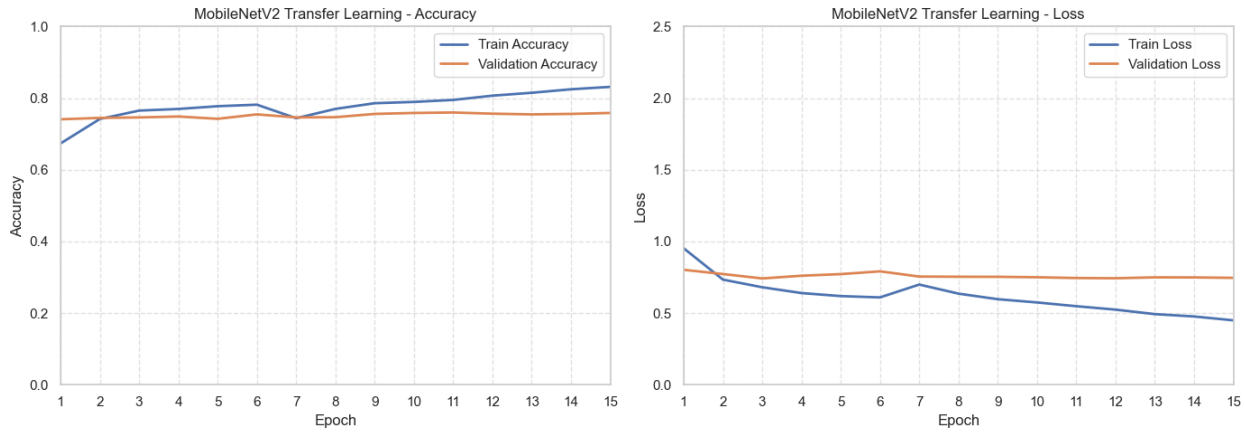


Figure 9: Training and validation accuracy and loss spanning both the initial training phase and the subsequent fine-tuning phase.

This fine-tuning step yielded the absolute best results of the entire project:

- **Final training accuracy:** 84.00% (Loss: 0.42)
- **Final validation accuracy:** 75.67% (Loss: 0.74)

(Note on Loss: The relatively high validation loss is normal when using Categorical Crossentropy. Because this function measures how far predicted probabilities are from the true class, it heavily penalizes the model for a lack of confidence, not just strict correctness).

V. Conclusion

Throughout this project, multiple architectures were developed and evaluated to classify Flickr images into five distinct categories. The models progressed from standard dense networks to custom convolutional architectures, and finally to advanced transfer learning.

	Model	# Train Accuracy	# Validation Accuracy	# Train Loss	# Validation Loss	# Parameters
0	MobileNetV2 Transfer Learning	0.84	0.76	0.42	0.74	2264389
1	CNN + GlobalAveragePooling2D	0.62	0.61	1.02	1.05	28357
2	Augmented Best CNN	0.6	0.58	1.06	1.09	28357
3	CNN + Dropout	0.64	0.58	0.85	1.09	125980357
4	Baseline CNN	0.64	0.57	0.87	1.12	125980357
5	MLP 2	0.55	0.53	1.16	1.18	12624645
6	CNN + Batch Normalization	0.55	0.53	1.16	1.18	125980741
7	MLP 1	0.62	0.51	1.01	1.17	25168901

Table 2: Comprehensive summary of all evaluated models, detailing their parameter counts, final training metrics, and final validation metrics.

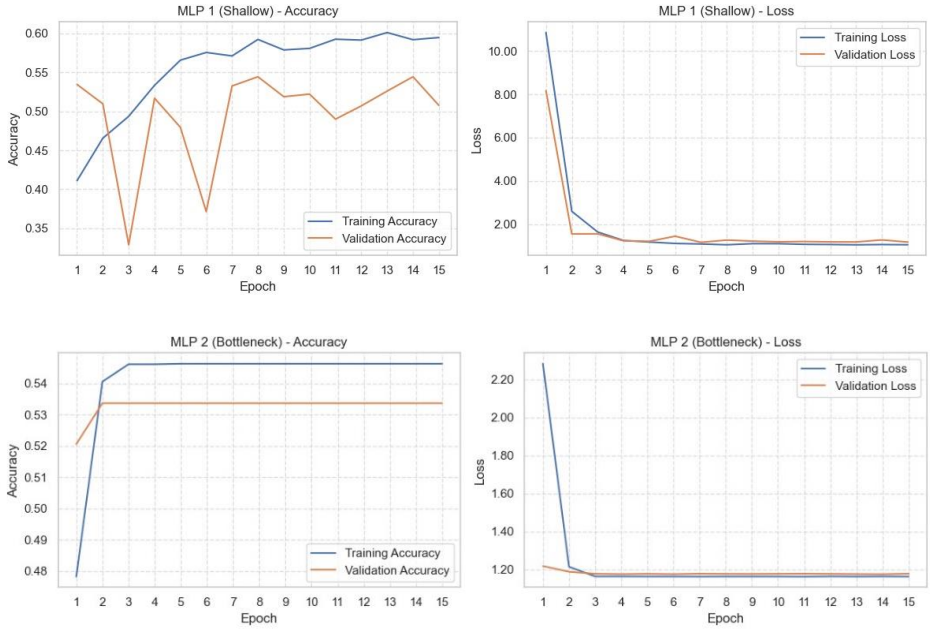
As demonstrated in the summary table above, the progression of experiments yields the following key takeaways:

- **MLPs lack spatial awareness:** Flattening images for standard MLPs caps predictive performance (max ~54% accuracy) and leads to overfitting.
- **Standard CNNs overfit:** Using a Flatten layer creates a massive parameter bottleneck (125M+ parameters), causing the model to memorize rather than generalize.
- **Global average pooling is crucial:** Replacing Flatten with GlobalAveragePooling2D slashes parameter count by 99.9%, mitigating overfitting and boosting custom CNN accuracy to over 61%.
- **Transfer learning is superior:** Pre-trained models drastically outperform custom architectures. MobileNetV2 was the clear winner, peaking at 76.00% validation accuracy.

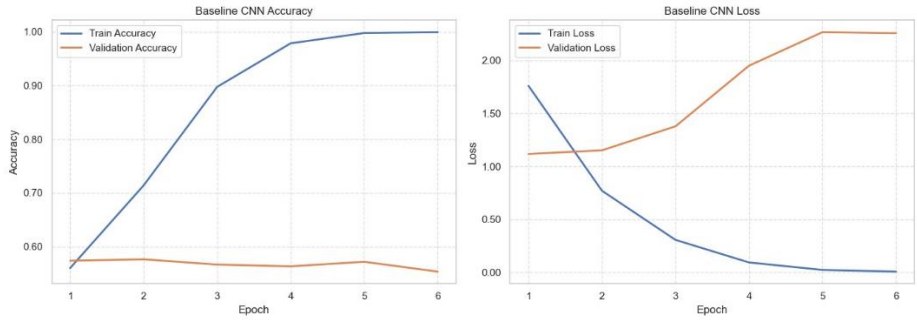
For complex image classification tasks, managing parameter counts through intelligent pooling and leveraging pre-trained networks (like MobileNetV2) are the most effective strategies for maximizing both accuracy and efficiency.

APPENDIX - Unscaled performance graphs

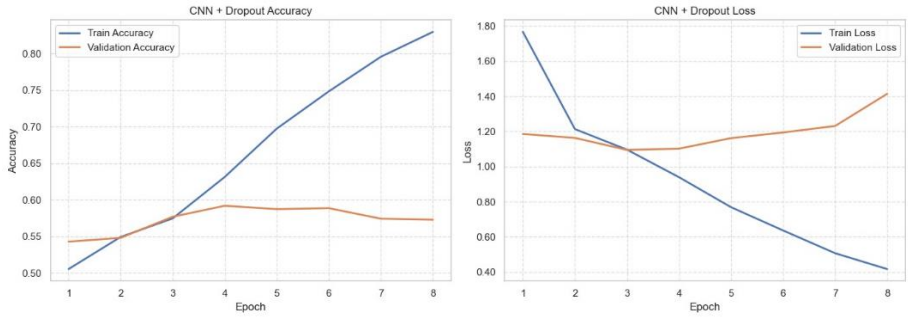
- Figure A1: Unscaled MLP 1 (Shallow) - Accuracy and Loss



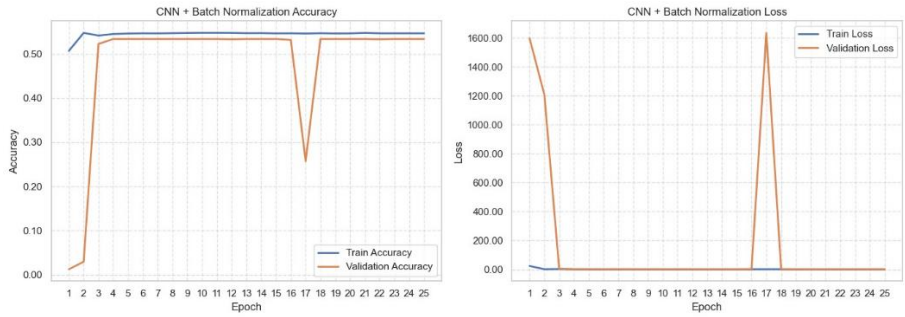
- Figure A2: Training and validation accuracy and loss for the Baseline CNN.



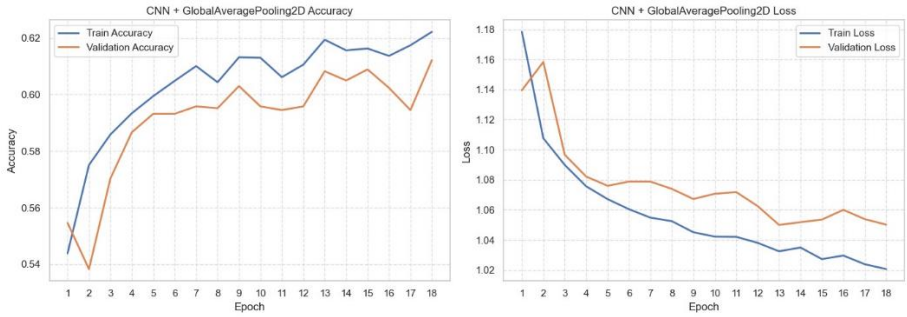
- Figure A3: Training and validation accuracy and loss for the CNN with Dropout.



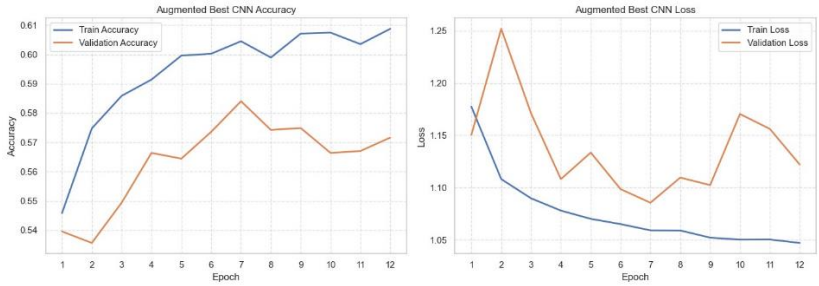
- Figure A4: Training and validation accuracy and loss for the CNN with Batch Normalization.



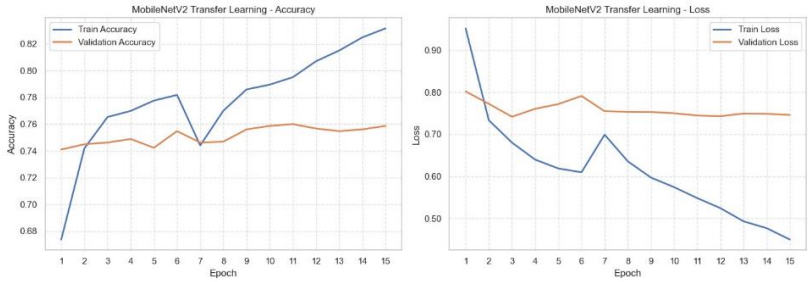
- Figure A5:Training and validation accuracy and loss for the CNN with Global Average Pooling.



- Figure A6: Training and validation accuracy and loss for the augmented CNN.



- Figure A9: Training and validation accuracy and loss spanning both the initial training phase and the subsequent fine-tuning phase.



REFRENCES

AI Assistance & Originality Disclosure

Statement of originality: The research, data collection, experimental results, and core analytical conclusions presented in this report are the **exclusive work of the groups**. The fundamental narrative and technical insights were developed independently by the research group to ensure the integrity of the financial analysis.

Linguistic enhancement: Google's **Gemini 3 Flash** large language model was utilized solely as a sophisticated editing tool to enhance the professional tone, structural flow, and grammatical precision of the group's original text. While the AI assisted in the stylistic formatting and "polishing" of the manuscript, all AI-generated suggestions were critically reviewed and verified by the authors to ensure 100% alignment with the actual research findings and data logs.